CODEQUEST – LEETCODE PROGRESS TRACKER

A PROJECT REPORT

Submitted by

NITHYA SHREE S

(2116240701367)

in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**CODEQUEST - LEETCODE PROGRESS TRACKER**" is the bonafide work of **NITHYA SHREE S [2116240701367],** who carried out the individual project work on the Zoho Creator Low-Code platform for the course CB23F35 - Low-Code Application Development using Zoho Creator during the academic year 2026 - 2027, Semester 5, under the supervision.

SIGNATURE

Dr. Manoranjini J
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. J. Jinu Sophia
COURSE FACULTY
Associate Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination held on: _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, NITHYA SHREE S [2116240701367], hereby declare that the project report titled "CodeQuest - LeetCode Progress Tracker" submitted for the End Semester Examination of the course CB23F35 - Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report including its data model, forms, reports, dashboard, portal pages, Deluge scripts and workflows was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai

Date: _____

SIGNATURE OF THE STUDENT
NITHYA SHREE S

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal Dr. S.N. MURUGESAN, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. Manoranjini J, Head of the Department, Department of Computer Science and Engineering for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. J. Jinu Sophia, Associate Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Creator Education Program for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

NITHYA SHREE S

ABSTRACT

Practising coding problems on platforms such as LeetCode is a common way for engineering students to prepare for technical interviews, but the effort is usually recorded in scattered notes and spreadsheets. When the practice history is kept in this way, it is difficult to see how many problems have been solved, how much effort each problem needed, which solved problems should be revised again and how the students of a group are progressing. The learner depends on memory for revision, and there is no common measure that rewards harder problems more than easy ones.

CodeQuest – LeetCode Progress Tracker is designed as a centralised low-code application using Zoho Creator and Deluge. It organises practice information through four linked forms: Problem_Bank, USER_PROFILE, Solution_Log and Revision_Tracker. Lookup fields connect every solution log entry and revision record to a student and to a problem. Workflows calculate the experience points of each attempt from the problem difficulty (Easy 10, Medium 20, Hard 30 and 0 for a failed submission), update the student's Total_XP and create a revision record seven days after a problem is solved successfully. Students enter their coding activity manually; the application does not integrate with LeetCode.

Four reports (All_User_Profiles, All_Problem_Banks, Solution_Log_Report and All_Revision_Trackers), a Solution Status Board in Kanban form and a Revision Calendar present the stored records. The CodeQuest_Dashboard shows the problems solved, the total XP and the total number of students, a difficulty distribution donut chart, a topic-wise bar chart and the recent student activity. A portal with Sign In, Sign Up, Reset Password and Create Password pages has been saved. Screenshots of the running application are included in this report, and values read from them agree with the XP and revision rules. Schedule, Blueprint and Approval features, record-level student access and production deployment evidence are not claimed.

Keywords: Coding Practice Tracking; Zoho Creator; Low-Code Development; Deluge Scripting; Workflow Automation; Gamification; Revision Scheduling.

PROJECT AT A GLANCE

Attribute	Details
Project Title	CodeQuest – LeetCode Progress Tracker
Application Domain	Coding practice tracking and student progress monitoring
Student Name & Register No.	[Student name and register number – to be entered]
Development Platform	Zoho Creator low-code platform with Deluge scripting
Application Scope	Student profiles, problem bank, solution log, revision tracker, XP calculation, revision scheduling, reports, Kanban board, calendar, dashboard and portal pages
Forms	4 core forms (Problem_Bank, USER_PROFILE, Solution_Log, Revision_Tracker)
Automation	3 workflows implemented with Deluge: XP calculation by difficulty, Total_XP update and revision record creation seven days after a solved problem
Process Control	Status-driven records (Solved / Failed solution status and revision status); no Schedule, Blueprint or Approval
Reports and Views	4 reports, a Solution Status Board (Kanban) and a Revision Calendar
Dashboard	3 KPI cards, 2 charts, a Recent Student Activity table and an All User Profiles navigation button
Portal	CodeQuest – LeetCode Progress Tracker portal with Sign In, Sign Up, Reset Password and Create Password pages saved
External Integration Points	None; students enter activity manually and there is no LeetCode API integration
Deployment Status	Application demonstrated in Zoho Creator (trial plan); published URL and production deployment evidence not supplied

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1	INTRODUCTION	12
1.1	Background	12
1.2	Problem Statement	12
1.3	Objectives	13
1.4	Scope of the Project	14
1.5	Justification for a Low-Code Approach	14
1.6	Organisation of the Report	14
2	LITERATURE SURVEY AND SYSTEM STUDY	15
2.1	Study of the Existing System	15
2.2	Limitations of the Existing System	15
2.3	Low-Code versus Traditional Development	16
2.4	Literature Survey	16
2.5	Emerging Trends in Low-Code Development	17
2.6	Proposed System	17
3	REQUIREMENT ANALYSIS	18
3.1	Stakeholders and User Roles	18
3.2	Functional Requirements	18
3.3	Non-Functional Requirements	19
3.4	Hardware and Software Requirements	19
3.5	Use Case Model	20
4	SYSTEM DESIGN	21
4.1	System Architecture	21
4.2	Data Model	21
4.3	Form Design	22
4.4	Process and Workflow Design	22
4.5	User Interface and Navigation Design	24
4.6	Access Control Design	25
5	IMPLEMENTATION	26
5.1	Application Setup	26
5.2	Forms and Data Capture	26
5.3	Reports	28
5.4	Pages and Dashboards	30

CHAPTER NO.	TITLE	PAGE NO.
5.5	Deluge Scripting	31
	Workflows and Automation	32
5.6		
5.7	Portal and Authentication Pages	32
6	TESTING AND DEBUGGING	33
6.1	Testing Strategy	33
6.2	Test Cases	33
6.3	Debugging Deluge Scripts	34
6.4	Defect Log	34
6.5	User Acceptance Testing	35
7	DEPLOYMENT, SECURITY AND GOVERNANCE	36
7.1	Environments and Deployment Strategy	36
7.2	Performance Optimisation	36
7.3	Security Features	37
7.4	Governance and Administration	37
8	RESULTS AND DISCUSSION	38
8.1	Application Output	38
8.2	Achievement of Objectives	38
8.3	Comparison with the Existing System	39
8.4	Limitations of the Low-Code Approach	39
8.5	Reflection on the No-Code → Low-Code → Pro-Code Spectrum	39
9	CONCLUSION AND FUTURE ENHANCEMENTS	40
9.1	Conclusion	40
9.2	Future Enhancements	40
9.3	Learning Outcomes and Career Relevance	40
10	REFERENCES	41
11	APPENDIX A – FEATURE COVERAGE CHECKLIST	42
12	APPENDIX B – REPORTS AND DASHBOARD CATALOGUE	43
13	APPENDIX C – DELUGE LOGIC SUMMARY	44
14	APPENDIX D – USER MANUAL	45
15	APPENDIX E – EVALUATION RUBRIC	46
16	APPENDIX F – VIVA-VOCE RECORD SHEET	48

LIST OF TABLES

Table 1.1: Project Objectives and Measures
Table 2.1: Low-Code and Traditional Development Comparison
Table 2.2: Literature Survey Summary
Table 3.1: Stakeholders and User Roles
Table 3.2: Functional Requirements
Table 3.3: Non-Functional Requirements
Table 3.4: Hardware and Software Requirements
Table 3.5: Use Case Descriptions
Table 4.1: Entities and Relationships
Table 4.2: XP Awarding Rules
Table 4.3: Access Control Design
Table 5.1: Forms Implemented
Table 5.2: Field Dictionary of the CodeQuest Forms
Table 5.3: Reports and Views Implemented
Table 5.4: Dashboard Components
Table 5.5: Solution Log Entries Visible in the Recent Student Activity Table
Table 5.6: Deluge Logic and Workflow Summary
Table 5.7: Portal Authentication Pages
Table 6.1: Test Cases
Table 6.2: Defect and Validation Log
Table 7.1: Deployment Checklist
Table 7.2: Security Controls Implemented
Table 8.1: Cross-Verification of Displayed Values with the XP and Revision Rules
Table 8.2: Objective Achievement Mapping
Table 8.3: Comparison with the Existing System
Table A.1: Feature Coverage Checklist
Table B.1: Scheduled Monitoring
Table D.1: Troubleshooting
Table E.1: Evaluation Rubric for the Individual Project
Table E.2: Performance Level Descriptors
Table F.1: Viva-Voce Questions and Responses
Table F.2: Marks Summary

LIST OF FIGURES

Figure 2.1: CodeQuest process represented as the proposed solution-tracking workflow

Figure 3.1: Primary actors and the CodeQuest application

Figure 4.1: Logical architecture of the CodeQuest application

Figure 4.2: Entity relationships between the four CodeQuest forms

Figure 4.3: XP calculation flowchart based on attempt status and problem difficulty

Figure 4.4: Solution submission workflow across Solution_Log, USER_PROFILE and Revision_Tracker

Figure 4.5: Revision scheduling flowchart for a solved Solution_Log entry

Figure 5.1: Problem Bank listing showing 20 of 20 problems with number, name, difficulty, topic and problem link

Figure 5.2: Solution Status Board (Kanban) grouping solution entries into Solved and Failed columns by student and problem

Figure 5.3: Revision Calendar for October 2026 showing scheduled revision entries by problem name

Figure 5.4: CodeQuest_Dashboard showing the Problems Solved, Total XP Earned and Total Students KPI cards with the Problem Difficulty Distribution donut chart

Figure 5.5: CodeQuest_Dashboard lower section showing the Topic-wise Solved Problems bar chart and the Recent Student Activity table of Solution Log Report entries

LIST OF ABBREVIATIONS

Abbreviation	**Expansion**
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
DSA	Data Structures and Algorithms
ER	Entity–Relationship
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
RBAC	Role-Based Access Control
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
XP	Experience Points

CHAPTER 1

INTRODUCTION

1.1 Background

Solving problems on coding practice platforms such as LeetCode is a common way for engineering students to prepare for technical interviews and to strengthen their command of data structures and algorithms. A learner works through many problems across topics such as arrays, hash maps, linked lists, trees, graphs, sliding windows and dynamic programming. The platform records the learner's own submissions, but it does not give a mentor or a study group a single view of who has solved what, how many attempts were needed and which solved problems should be revised again.

Learners therefore keep their own notes in notebooks or spreadsheets. Revision of an older problem depends on the learner remembering to return to it, and there is no consistent reward that encourages regular practice. A centralised and partly automated tracker can reduce this effort by capturing each entry once, awarding experience points by difficulty and scheduling a revision of every successfully solved problem.

CodeQuest – LeetCode Progress Tracker is a low-code application developed on the Zoho Creator platform using Deluge scripting. It stores student profiles, a bank of practice problems, a log of solution attempts and a tracker of revisions. Workflows calculate experience points (XP) from problem difficulty, update each student's total XP and create a revision record seven days after a problem is solved successfully. Reports, a Kanban status board, a revision calendar and a dashboard allow the progress of the students to be reviewed at a glance. Students enter their coding activity manually; the application does not connect to LeetCode.

1.2 Problem Statement

Tracking competitive programming practice through scattered notes and spreadsheets makes it difficult to monitor consistent progress, to remember when a solved problem should be revised and to compare effort across students. The absence of a single consolidated record reduces the motivation and the discipline needed for steady practice.

- Solved problems, failed attempts and personal approach notes are stored in different places, so there is no single log of practice history.

- There is no uniform measure of effort: an easy problem and a hard problem are counted alike, and failed attempts are not distinguished from successful ones.
- Revision depends on memory. A problem that was solved once is rarely revisited, so the learning is easily forgotten.
- A mentor or group has no quick view of the number of problems solved, the XP earned, the topics practised or the most recent activity of each student.

1.3 Objectives

- Design linked forms for student profiles, a shared problem bank, a solution log and a revision tracker.
- Automate XP calculation from problem difficulty (Easy 10, Medium 20, Hard 30, failed submission 0) and keep each student's Total_XP up to date.
- Create a revision record automatically seven days after a problem is solved successfully.
- Provide reports, a Kanban status board and a revision calendar for reviewing solutions and revisions.
- Provide a dashboard with problems solved, total XP, total students, difficulty distribution, topic-wise progress and recent student activity.
- Configure the portal authentication pages (Sign In, Sign Up, Reset Password and Create Password).

Objective ID	Objective	Evidence / Output
OBJ-01	Design linked forms for profiles, problems, solutions and revisions.	Four forms with lookup fields (Solution_Log and Revision_Tracker)
OBJ-02	Calculate XP by difficulty and update Total_XP.	XP workflows; XP_Earned and Total_XP fields; Fig. 5.4
OBJ-03	Create a revision record seven days after a solved entry.	Revision_Tracker records; Revision Calendar (Fig. 5.3)
OBJ-04	Provide reports, Kanban board and revision calendar.	Four reports, Solution Status Board and Revision Calendar
OBJ-05	Provide a monitoring dashboard.	CodeQuest_Dashboard (Figs. 5.4 and 5.5)
OBJ-06	Configure portal authentication pages.	Four saved portal pages (Section 5.7)

Table 1.1: Project Objectives and Measures

1.4 Scope of the Project

The scope includes the student profile, a problem bank, a solution log in which every attempt is recorded, and a revision tracker. It also includes XP calculation, the update of the student's total XP, automatic revision records, four reports, a Kanban status board, a revision calendar, the CodeQuest_Dashboard and four portal authentication pages.

Students enter their coding activity manually. The application does not call the LeetCode API, does not read submissions from LeetCode and does not verify that a problem was actually solved on the platform; the Problem_link field only stores the address of the problem for convenience. Schedule, Blueprint and Approval features are not part of the completed implementation. Complete student-specific portal access and record-level permissions are not claimed. The final published environment is treated as an item to be verified, because production deployment details were not supplied.

1.5 Justification for a Low-Code Approach

Zoho Creator provides visual form building, lookups, reports, dashboards, workflows, portals and Deluge scripting within one platform. CodeQuest is a data-centred application whose requirements consist of data capture, lookups, simple calculations, record creation, status views and summary indicators, and these fit the platform well. A change to a field, report or workflow can be made without rebuilding the application. Platform capabilities and limitations are discussed with reference to the official Zoho Creator documentation [1].

1.6 Organisation of the Report

Chapter 2 studies the existing way of tracking coding practice and the proposed solution. Chapter 3 presents stakeholders and requirements. Chapter 4 describes the architecture, data relationships and process design. Chapter 5 documents the forms, reports, dashboard, workflows and portal pages with screenshots of the running application. Chapter 6 presents the test approach and the test cases. Chapter 7 discusses deployment, security and governance. Chapter 8 reviews the results and limitations, and Chapter 9 concludes the report and identifies future enhancements.

CHAPTER 2

LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the commonly followed approach, a student solves problems on LeetCode and keeps personal notes in a notebook or spreadsheet. The platform shows the student's own solved count, but it does not store the learner's approach notes in a form that a mentor can review, it does not combine the progress of several students, and it does not remind the learner to revise an older problem. Where a group or a classroom wants to compare progress, the information has to be collected by hand and consolidated in a separate sheet.

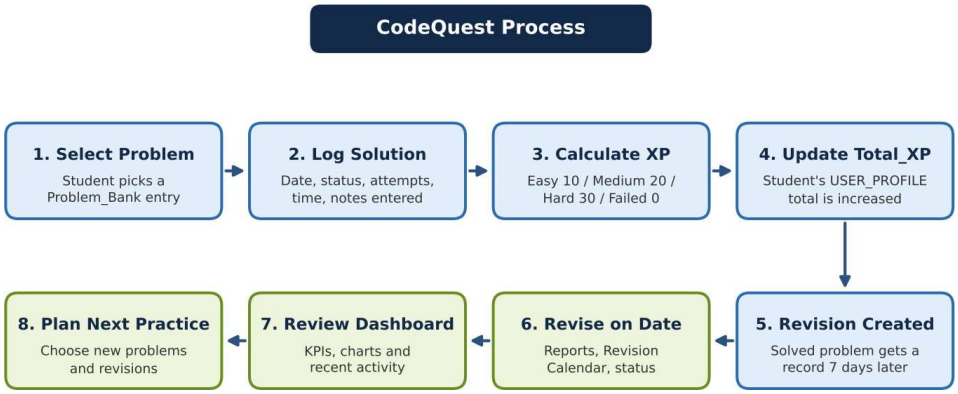


Figure 2.1: *CodeQuest process represented as the proposed solution-tracking workflow*

2.2 Limitations of the Existing System

- Practice history is fragmented across notes, spreadsheets and the platform profile.
- There is no common measure of effort that rewards difficult problems more than easy ones.
- Revision of solved problems is not scheduled and depends on the learner's memory.
- A mentor cannot see the number of problems solved, the XP earned or recent activity of all students in one place.
- Topic-wise strengths and weaknesses are not summarised automatically.
- Problem details are repeated in every personal record and are easily mistyped.

2.3 Low-Code versus Traditional Development

Criterion	Traditional Pro-Code Development	Low-Code on Zoho Creator
Development effort	Requires a custom user interface, database layer, APIs and deployment setup.	Uses visual form, report and page builders with configured workflows.
Data capture	Forms and validation are coded individually.	Forms, dropdowns, lookups and mandatory fields are configured visually.
Automation	Requires application logic to calculate values and create records.	Workflows and Deluge scripts support event-driven calculation and record creation.
Reports and views	Charts, boards and calendars need separate libraries.	Reports, Kanban and calendar views and dashboard charts are configured in the platform.
Authentication	Login, registration and password reset must be built and secured.	Portal sign-in, sign-up and password pages are provided by the platform.
Hosting	Infrastructure and release process are managed separately.	Platform hosting and application publishing are provided by the service.
Customisation	Offers broad control over architecture and UI.	Customisation is bounded by platform capabilities and plan limits.
Maintenance	Requires code-level maintenance and deployment discipline.	Routine changes are simpler, but scripts and permissions still need testing.

Table 2.1: Low-Code and Traditional Development Comparison

2.4 Literature Survey

The following sources provide the platform and process background for this project. The project brief does not include experimental results; the table therefore focuses on platform documentation, established work on gamification and distributed practice, and a security reference, and does not claim results from experiments that were not carried out.

S.No.	Source	Focus	Relevance
1	Zoho Creator documentation [1]	Forms, reports, pages and application building	Primary reference for platform capabilities.
2	Zoho Deluge documentation [2]	Scripting language, fetch, insert and update tasks	Supports XP calculation, Total_XP update and revision record creation.
3	Zoho Creator workflow documentation [3]	Event-based form workflows	Relevant to the Solution_Log workflows.
4	Zoho Creator reports documentation [4]	List, Kanban and calendar reports	Relevant to the Solution Status Board and Revision Calendar.
5	Zoho Creator pages documentation [5]	Dashboard pages, KPI cards and charts	Relevant to CodeQuest_Dashboard.
6	Zoho Creator portal documentation [6]	Portals and authentication pages	Relevant to the Sign In, Sign Up, Reset Password and Create Password pages.

S.No.	Source	Focus	Relevance
7	Deterding et al. [7]	Game design elements in non-game contexts	Background for the XP-based reward element.
8	Cepeda et al. [8]	Distributed (spaced) practice	Background for scheduling a later revision of solved problems.
9	OWASP ASVS [9]	Application security controls	General checklist for access and data protection.

Table 2.2: Literature Survey Summary

2.5 Emerging Trends in Low-Code Development

Low-code platforms increasingly combine visual modelling, reusable components, event-driven automation, portals and analytics. For CodeQuest, these patterns support a small relational data model, automatic calculation of XP and revision records, and dashboard reporting. Two ideas from educational technology are also relevant. Gamification, the use of game design elements such as points in a non-game setting [7], is reflected in the XP score. Distributed practice, in which review is spread over time rather than concentrated in one session [8], motivates the revision record that is created after a problem is solved. The project applies these ideas in a simple form and does not claim measured learning gains. Automated rules still need careful design: workflows require clear handling of edits, protection against duplicate records, role-appropriate permissions and testing with valid and invalid entries.

2.6 Proposed System

The proposed system centralises practice information in four linked Zoho Creator forms. The Problem_Bank holds each problem once. A student's attempt is stored in Solution_Log with lookups to the USER_PROFILE and the Problem_Bank. When a record is saved, the XP earned is calculated from the difficulty of the selected problem: 10 for Easy, 20 for Medium, 30 for Hard and 0 for a failed submission. The student's Total_XP is updated, and a Revision_Tracker record is created seven days after a successfully solved problem. Four reports, a Kanban status board and a revision calendar present the stored records, and the CodeQuest_Dashboard summarises problems solved, XP, students, difficulty distribution, topic-wise progress and recent activity. A portal with four authentication pages provides the sign-in layer.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Role	Responsibilities
Student / Learner	Maintains profile details, selects problems from the Problem Bank, manually records each attempt with its date, status, attempts, time taken and approach notes, and reviews the revision calendar, reports and dashboard.
System (Workflows)	Calculates XP from problem difficulty, updates Total_XP and creates a revision record seven days after a successful solution.
Administrator / Developer	Maintains the Problem Bank, configures forms, reports, the dashboard, workflows and portal pages, reviews data, and publishes the application.

Table 3.1: Stakeholders and User Roles

3.2 Functional Requirements

ID	Requirement	Role(s)	Priority
FR-01	Maintain a student profile with name, e-mail, daily goal, target problems, user id and total XP.	Student, Admin	High
FR-02	Maintain a problem bank with problem number, name, difficulty, topic and link.	Admin	High
FR-03	Record a solution attempt linked to a student and a problem, with date, status, attempts, time taken and approach notes.	Student	High
FR-04	Calculate XP earned: Easy 10, Medium 20, Hard 30 and failed submission 0.	System	High
FR-05	Update the student's Total_XP after XP is calculated.	System	High
FR-06	Create a revision record seven days after a problem is solved successfully.	System	High
FR-07	Store the revision date and revision status for each revision record.	Student, System	Medium
FR-08	Provide the reports All_User_Profiles, All_Problem_Banks, Solution_Log_Report and All_Revision_Trackers.	All	High
FR-09	Provide a Solution Status Board (Kanban) grouping solutions by status.	All	Medium
FR-10	Provide a Revision Calendar showing scheduled revisions.	Student	Medium
FR-11	Show Problems Solved, Total XP and Total Students as KPI cards.	All	High
FR-12	Show a difficulty distribution donut chart and a topic-wise solved problems bar chart.	All	Medium
FR-13	Show recent student activity from the solution log on the dashboard.	All	Medium
FR-14	Provide a dashboard button that navigates to All User Profiles.	All	Low

ID	Requirement	Role(s)	Priority
FR-15	Provide portal Sign In, Sign Up, Reset Password and Create Password pages.	Student, Admin	Medium

Table 3.2: Functional Requirements

3.3 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Data integrity	Lookup relationships and mandatory fields should keep solution and revision records linked to valid students and problems.
NFR-02	Security and privacy	Student data should be accessible only to authenticated, permitted users; portal authentication pages support sign-in.
NFR-03	Usability	Forms, reports, boards and the dashboard should use consistent labels and a clear navigation menu.
NFR-04	Maintainability	Workflow logic should be named, documented and tested after every change.
NFR-05	Accuracy	XP values and totals should follow the difficulty rules exactly and match the stored records.
NFR-06	Availability	Application access depends on platform availability and network connectivity.
NFR-07	Performance	Reports and dashboard components should display the relevant records without unnecessary repeated queries.

Table 3.3: Non-Functional Requirements

3.4 Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator low-code application platform
Scripting	Deluge for workflow logic, calculation and record creation
Client	Desktop, laptop or mobile device with a modern browser and a stable internet connection
Browser	Current version of Google Chrome, Microsoft Edge or Mozilla Firefox
Development	Zoho Creator form, report, page, workflow and portal builders
Data entry	Manual entry by students through the application forms; no automatic import from LeetCode
Regional settings	Dates are displayed in the dd-MMM-yyyy format (for example 01-Oct-2026) in the Solution Log Report

Table 3.4: Hardware and Software Requirements

3.5 Use Case Model

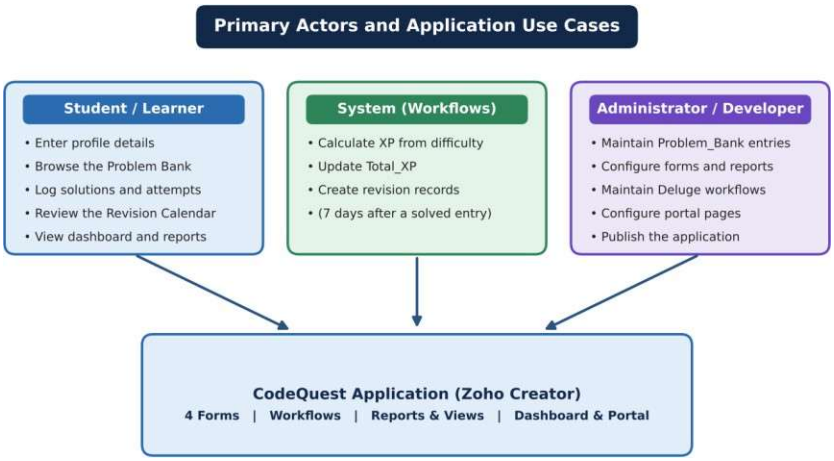


Figure 3.1: *Primary actors and the CodeQuest application*

Use Case	Precondition	Main flow	Result
Add problem	Administrator has the problem details.	Open the Problem Bank form, enter number, name, difficulty, topic and link, then submit.	Problem is available in lookups and reports.
Log solved problem	Student profile and problem exist.	Open Solution_Log, select student and problem, enter date, status Solved, attempts, time and notes.	XP is calculated, Total_XP is updated and a revision record is created.
Log failed attempt	Student profile and problem exist.	Enter the attempt with status Failed.	XP_Earned is 0 and no seven-day revision record is created.
Review revisions	Revision records exist.	Open the Revision Calendar or the All_Revision_Trackers report.	Student sees problems due for revision by date.

Table 3.5: Use Case Descriptions

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The architecture separates the user-facing layer, the workflow logic and the stored application records, as shown in Figure 4.1. Students and the administrator interact with Zoho Creator forms, reports, the dashboard page and the portal pages. Workflows written in Deluge calculate XP, update the student's total and create revision records. The linked records provide the operational data used by the reports, the Kanban board, the revision calendar and the dashboard.

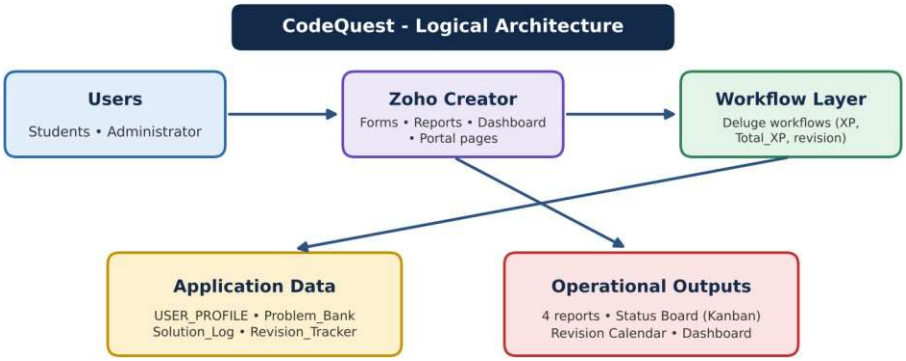


Figure 4.1: *Logical architecture of the CodeQuest application*

4.2 Data Model

The data model contains two master forms and two transaction forms. USER_PROFILE and Problem_Bank are the master forms: each student and each problem is stored once. Solution_Log and Revision_Tracker are transaction forms that refer to both masters through lookup fields, so that the student's name and the problem's name are displayed from the master records instead of being typed again. One student can have many solution log entries and many revision records, and one problem can appear in the records of many students. Figure 4.2 shows these relationships and Table 4.1 summarises the purpose of each entity.

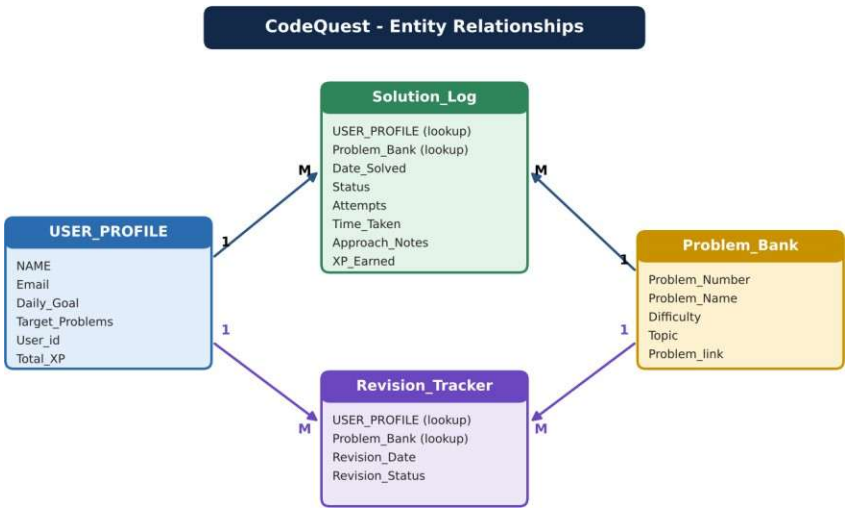


Figure 4.2: *Entity relationships between the four CodeQuest forms*

Entity	Relationship / Purpose
USER_PROFILE	Stores the student's name, e-mail, daily goal, target problems, user id and Total_XP. One student can be referred to by many Solution_Log and Revision_Tracker records.
Problem_Bank	Stores the problem number, name, difficulty, topic and link of each practice problem. The difficulty determines the XP awarded, and the topic is used in the topic-wise chart.
Solution_Log	Stores each attempt with lookups to USER_PROFILE and Problem_Bank, the date solved, status, attempts, time taken, approach notes and the XP earned.
Revision_Tracker	Stores a scheduled revision with lookups to USER_PROFILE and Problem_Bank, the revision date and the revision status.

Table 4.1: Entities and Relationships

4.3 Form Design

Problem_Bank is the shared reference form and holds the difficulty and topic used for XP calculation and charts. USER_PROFILE holds the student's identity, goals and the cumulative Total_XP maintained by the workflow. Solution_Log records every attempt through two lookups and a status field that separates Solved from Failed entries, and Revision_Tracker holds the revision date and revision status with lookups to the student and the problem.

4.4 Process and Workflow Design

CodeQuest controls its records through status fields and workflows rather than through a Blueprint or an approval process. The status of a solution log entry distinguishes a solved attempt from a failed one; the Solution Status Board in Figure 5.2 shows these two groups. Revision_Tracker carries a revision status that records the state of each revision. A Blueprint

was not required, because the transitions are not restricted by role; it could be introduced later if staged review by a mentor is added.

XP calculation

The XP earned by an attempt depends on the difficulty of the problem selected through the Problem_Bank lookup. Table 4.2 lists the rules and Figure 4.3 shows the decision sequence.

Condition	XP awarded	Explanation
Easy problem solved	10	The lowest reward, for problems with Difficulty set to Easy.
Medium problem solved	20	Twice the Easy reward, for problems with Difficulty set to Medium.
Hard problem solved	30	The highest reward, for problems with Difficulty set to Hard.
Failed submission	0	No XP is awarded when the attempt status is Failed.

Table 4.2: XP Awarding Rules

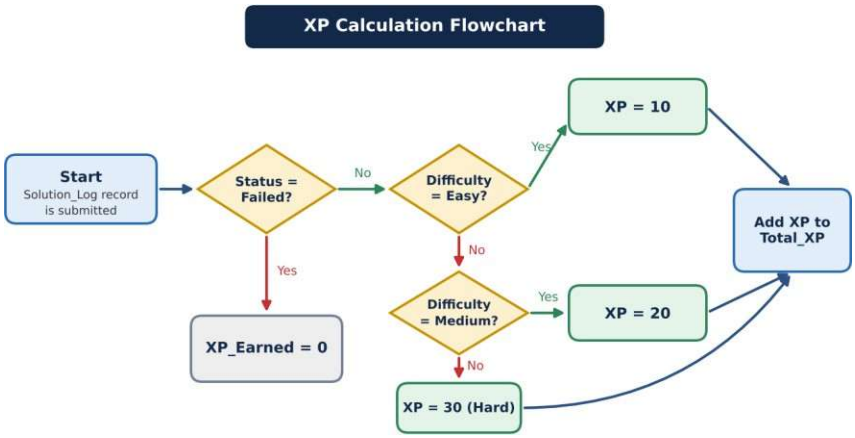


Figure 4.3: *XP calculation flowchart based on attempt status and problem difficulty*

Solution submission

When a student submits a Solution_Log record, the workflow reads the difficulty from the selected problem, stores the XP in the XP_Earned field and adds it to the student's Total_XP. If the attempt is solved, a revision record is created. Figure 4.4 presents this flow between the forms.

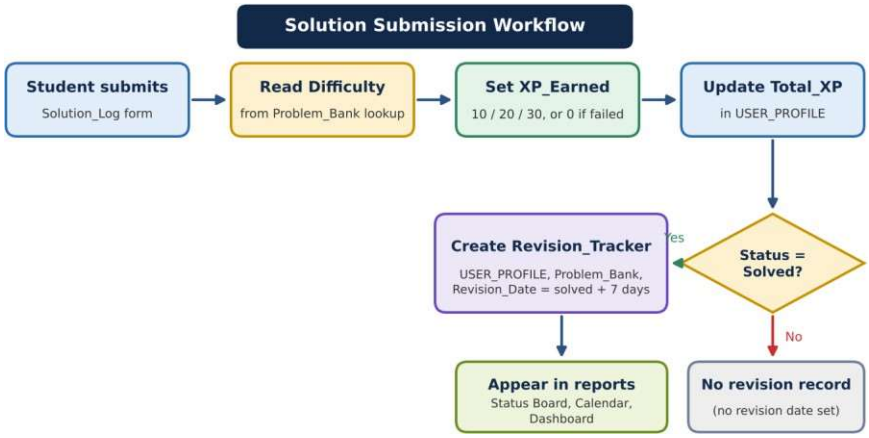


Figure 4.4: *Solution submission workflow across Solution_Log, USER_PROFILE and Revision_Tracker*

Revision scheduling

The revision date is calculated as the Date_Solved plus seven days and is stored in a new Revision_Tracker record together with the student and the problem, as shown in Figure 4.5. A failed attempt does not create a seven-day revision record. The record appears in the All_Revision_Trackers report and on the Revision Calendar.

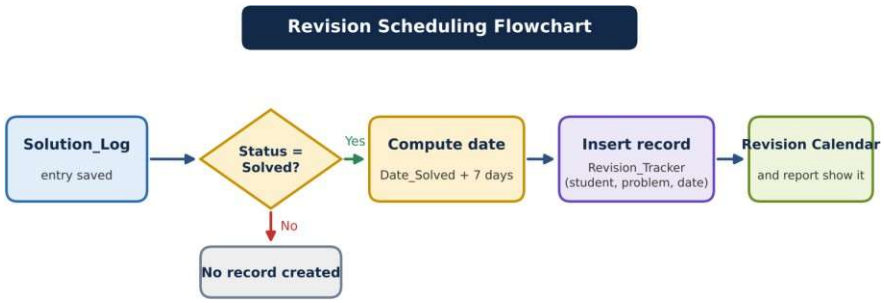


Figure 4.5: *Revision scheduling flowchart for a solved Solution_Log entry*

4.5 User Interface and Navigation Design

The application is organised around a left navigation menu with one section for each area: User Profiles, Problem Banks, Solution Log, Revision Trackers and the CodeQuest_Dashboard. The Problem Banks section contains the Problem Bank form and its listing views. The Solution Log section contains the Solution Log form, the Solution Log Report and the Solution Status Board. The Revision Trackers section contains the Revision Tracker form, the All Revision Trackers report and the Revision Calendar. The header of every screen shows the application name, CodeQuest – LeetCode Progress Tracker, and the signed-in user is displayed at the bottom of

the menu. The dashboard opens with three KPI cards followed by the charts and the recent activity table, and a button navigates to All User Profiles.

4.6 Access Control Design

Access control in Zoho Creator is based on profiles and, for portals, on portal user roles. The supplied project evidence does not include the share settings of the application. The permissions described in Table 4.3 therefore follow the default profile descriptions of the platform; no custom field-level or record-level restrictions are claimed, and complete student-specific portal access has not been demonstrated.

Role / Profile	Typical permitted actions (platform default)	Status in CodeQuest
Administrator	Full access to all components, configuration and permissions.	Used by the application owner for configuration and review.
Developer	Build and modify forms, workflows, reports and pages.	Used during development; changes should be tested before publishing.
Write	Add, edit and view records in all components.	Custom assignment not verified.
Read	View records and reports in all components.	Custom assignment not verified.
Portal user	Sign in through the portal pages.	Pages saved; record-level restriction by student not claimed.

Table 4.3: Access Control Design

CHAPTER 5

IMPLEMENTATION

5.1 Application Setup

The application is named CodeQuest – LeetCode Progress Tracker and was built on Zoho Creator using the visual builders for forms, reports, pages and workflows. The screenshots in this report were captured from the running application in live mode, where the Zoho Creator header shows that the application was being used on a trial plan. Form link names such as USER_PROFILE, Problem_Bank, Solution_Log and Revision_Tracker are used consistently because Deluge scripts refer to field link names. Before publishing changes, the developer should verify field link names, lookup targets, mandatory fields and workflow order.

5.2 Forms and Data Capture

The application contains four forms, summarised in Table 5.1. The purpose and function of every field are listed in Table 5.2, and the four lookup fields (USER_PROFILE and Problem_Bank in both Solution_Log and Revision_Tracker) are marked in the table. Figure 5.1 shows the Problem Bank listing, which displays 20 of 20 stored problems.

No.	Form	Purpose and principal fields
1	Problem_Bank	Shared list of practice problems: Problem_Number, Problem_Name, Difficulty, Topic and Problem_link.
2	USER_PROFILE	Student details and progress totals: NAME, Email, Daily_Goal, Target_Problems, User_id and Total_XP.
3	Solution_Log	Record of each attempt: USER_PROFILE, Problem_Bank, Date_Solved, Status, Attempts, Time_Taken, Approach_Notes and XP_Earned.
4	Revision_Tracker	Scheduled revisions: USER_PROFILE, Problem_Bank, Revision_Date and Revision_Status.

Table 5.1: Forms Implemented

Form	Field	Function
Problem_Bank	Problem_Number	The LeetCode problem number (for example 76, 704 and 1025); identifies the problem and is shown first in the listing.
Problem_Bank	Problem_Name	The title of the problem; displayed in lookups, the Kanban cards and the revision calendar.
Problem_Bank	Difficulty	Easy, Medium or Hard; determines the XP awarded and the difficulty distribution chart.
Problem_Bank	Topic	The topic of the problem (for example Array, Hash Map, Graph or Sliding Window); used by the topic-wise chart.

Form	Field	Function
Problem_Bank	Problem_link	Web address of the problem on LeetCode, shown as a link in the listing.
USER_PROFILE	NAME	The student's name, displayed in lookups and on the Kanban board.
USER_PROFILE	Email	The student's e-mail address, stored as part of the profile.
USER_PROFILE	Daily_Goal	The number of problems the student intends to solve each day; stored as profile data.
USER_PROFILE	Target_Problems	The total number of problems the student aims to solve; stored as profile data.
USER_PROFILE	User_id	An identifier for the student.
USER_PROFILE	Total_XP	Cumulative XP of the student; increased by the workflow after each calculation.
Solution_Log	USER_PROFILE	Lookup to the student who made the attempt.
Solution_Log	Problem_Bank	Lookup to the problem that was attempted; its difficulty drives the XP.
Solution_Log	Date_Solved	The date of the entry, shown as dd-MMM-yyyy; also the base date for the seven-day revision.
Solution_Log	Status	Outcome of the attempt; the Kanban board shows the values Solved and Failed.
Solution_Log	Attempts	The number of attempts the student needed.
Solution_Log	Time_Taken	The time spent on the problem, entered as a number by the student.
Solution_Log	Approach_Notes	The student's own notes about the approach (for example "Used HashMap").
Solution_Log	XP_Earned	The XP awarded for the attempt, calculated from the difficulty (0 when failed).
Revision_Tracker	USER_PROFILE	Lookup to the student who has to revise.
Revision_Tracker	Problem_Bank	Lookup to the problem to be revised; its name is shown on the calendar.
Revision_Tracker	Revision_Date	The date on which the revision is scheduled.
Revision_Tracker	Revision_Status	The state of the revision, used to follow whether it has been done.

Table 5.2: Field Dictionary of the CodeQuest Forms

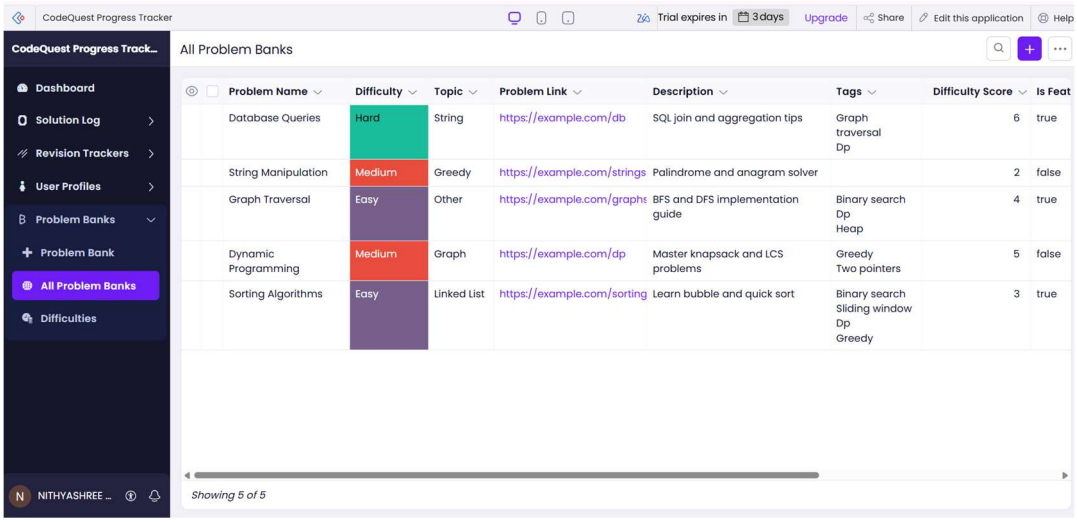


Figure 5.1: *Problem Bank*

The listing in Figure 5.1 includes problems such as Minimum Window Substring (76, Hard, Sliding Window), Binary Search (704, Easy, Binary Search) and Number of Islands (200, Medium, Graph). The topics visible in the screenshot include Sliding Window, Dynamic Programming, Binary Search, Graph, Hash Map, Tree, Heap, Linked List and Stack, and every problem carries a link to its page on LeetCode.

5.3 Reports

Four reports present the stored records: All_User_Profiles, All_Problem_Banks, Solution_Log_Report and All_Revision_Trackers. Two further views were configured on the same data: the Solution Status Board, a Kanban view of the solution log, and the Revision Calendar. Table 5.3 summarises them.

No.	Report / View	Source form	Use
1	All_User_Profiles	USER_PROFILE	Lists all students with their goals and Total_XP.
2	All_Problem_Banks	Problem_Bank	Lists all problems with difficulty, topic and link.
3	Solution_Log_Report	Solution_Log	Lists attempts with student, date, status, attempts, time, notes and problem.
4	All_Revision_Trackers	Revision_Tracker	Lists scheduled revisions with student, problem, date and status.
5	Solution Status Board (Kanban)	Solution_Log	Shows solution entries as cards grouped by status.
6	Revision Calendar	Revision_Tracker	Shows revision records on a month, week or day calendar.

Table 5.3: Reports and Views Implemented

The Solution Status Board in Figure 5.2 groups the entries into a Solved column and a Failed column. Each card shows the student's name and the problem name, for example Harshini with Top K Frequent Elements in the Solved column and Kaviya with Coin Change in the Failed column. The cards of three students (Harshini, Kaviya and Arjun) are visible on the board.

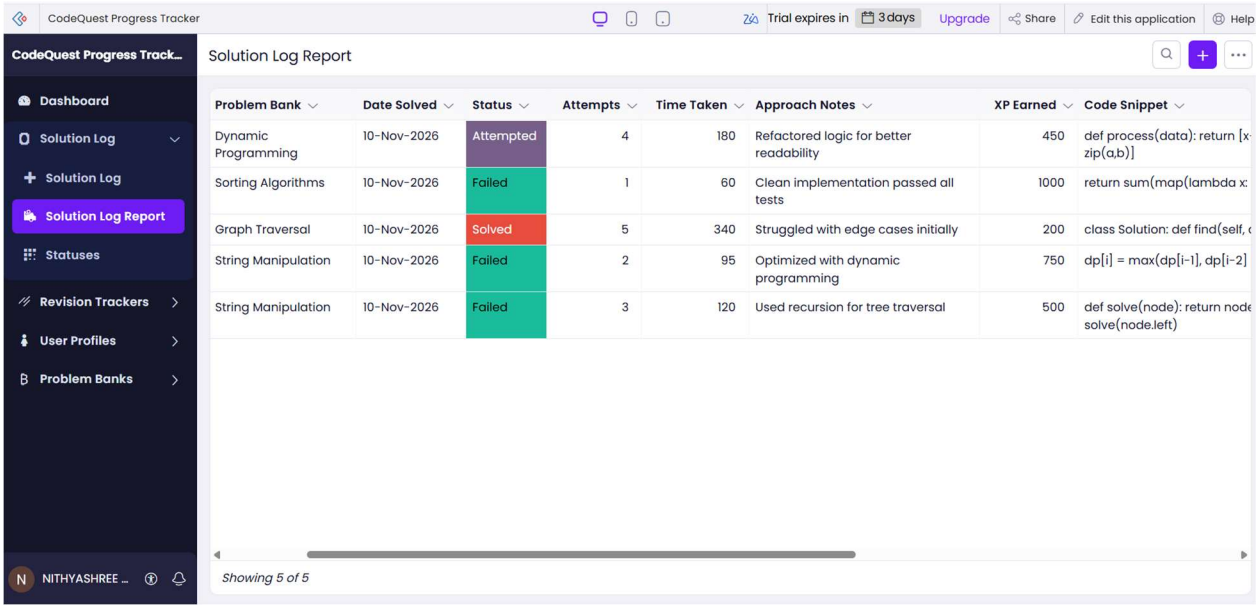


Figure 5.2: *Solution Log Report*

The Revision Calendar in Figure 5.3 displays the month of October 2026 with the Month, Week and Day options. Each scheduled revision appears on its date as a bar carrying the problem name; for example, Best Time to Buy and Sell Stock, Climbing Stairs and Two Sum are shown on Thursday, 8 October, and Valid Parentheses is shown on Friday, 9 October.

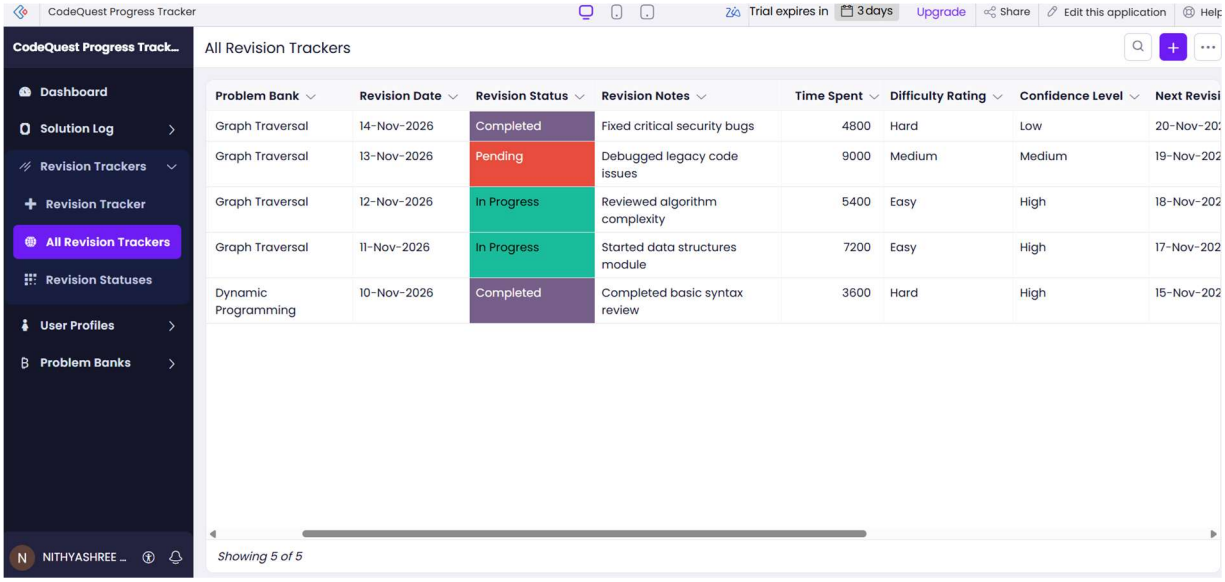


Figure 5.3: *Revision Tractor*

5.4 Pages and Dashboards

The CodeQuest_Dashboard is a page that combines indicator cards, charts and an embedded report. Table 5.4 lists its components, and Figures 5.4 and 5.5 show the dashboard as captured from the running application.

No.	Dashboard component	Description
1	Problems Solved KPI	Shows the number of solved problems; the captured value is 29.
2	Total XP KPI	Shows the total XP earned; the captured value is 420.
3	Total Students KPI	Shows the number of students; the captured value is 3.
4	Problem Difficulty Distribution	Donut chart of solved problems by difficulty, described as a breakdown across Easy, Medium and Hard levels.
5	Topic-wise Solved Problems	Horizontal bar chart of problem counts by topic.
6	Recent Student Activity	Embeds the Solution Log Report, with the subtitle “Latest problem-solving submissions from students”.
7	All User Profiles button	Navigation button that opens All User Profiles; not visible in the supplied screenshots.

Table 5.4: Dashboard Components

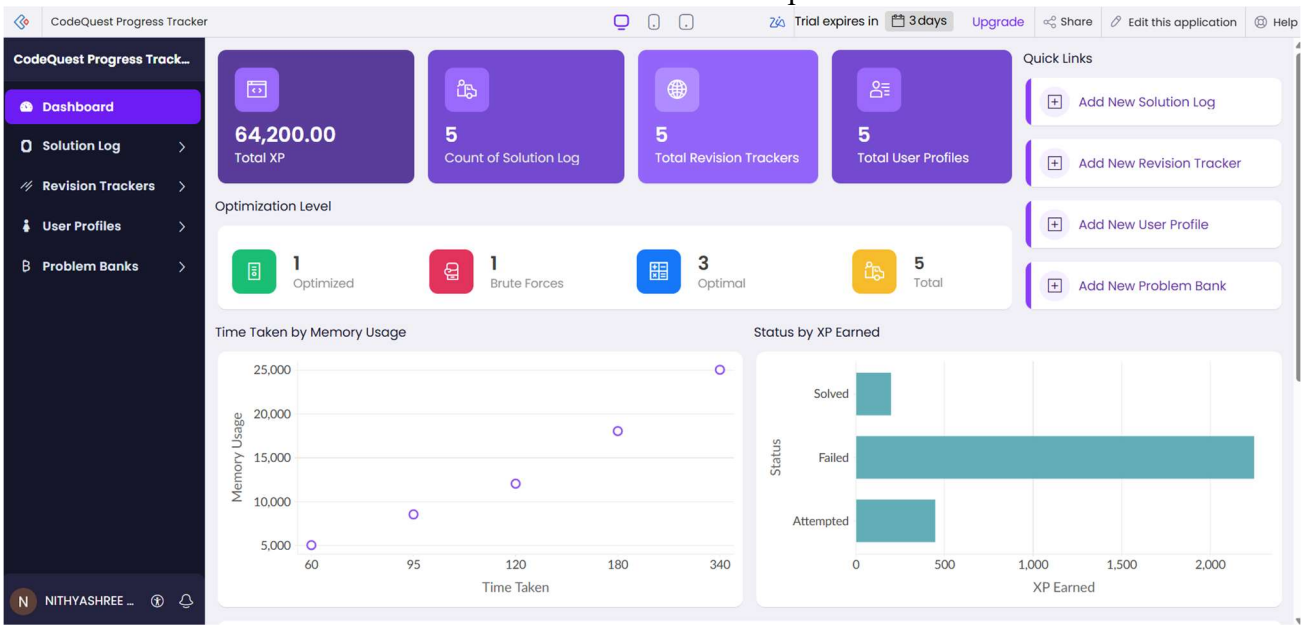


Figure 5.4: *CodeQuest_Dashboard*

In Figure 5.4, the three KPI cards display 29 problems solved, 420 XP earned and 3 students. The donut chart uses a legend with Easy and Medium and labels its two segments 16 and 13. The two segments add up to the 29 solved problems, and 16 Easy problems at 10 XP plus 13 Medium problems at 20 XP give exactly 420 XP, which agrees with the XP rule and the Total XP card (Section 8.1). No Hard segment appears in the legend of the captured chart.

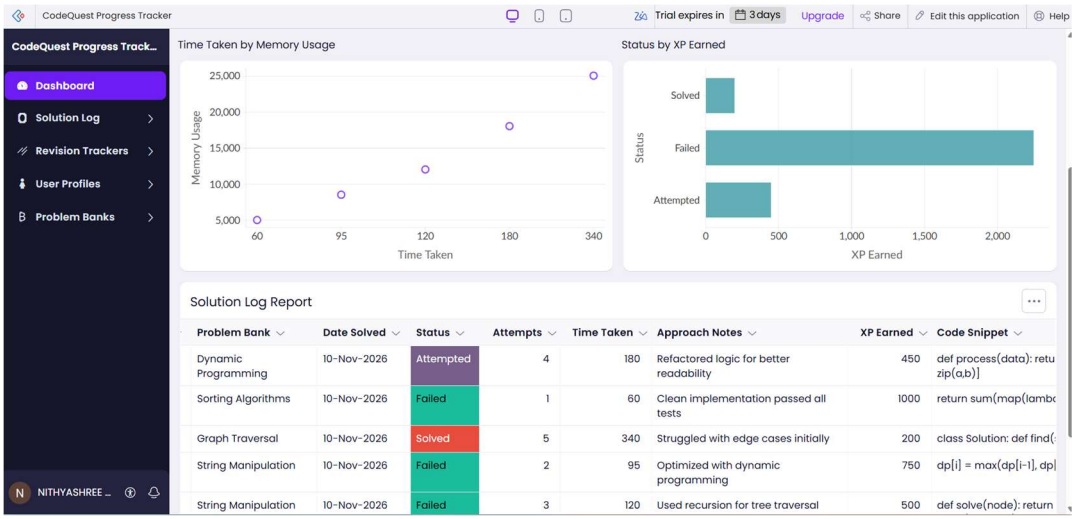


Figure 5.5: *CodeQuest_Dashboard lower section*

Figure 5.5 shows the topic-wise bar chart, in which the topics are listed on the vertical axis and the counts on the horizontal axis. In the captured chart, Array and Dynamic Programming have a count of 5, Hash Map has 3 and Tree has 1, and a tooltip shows a count of 5 for Graph. The Recent Student Activity section embeds the Solution Log Report with the columns USER PROFILE, Date Solved, Status, Attempts, Time Taken, Approach/Notes and Problem Bank. Table 5.5 reproduces the rows visible in the screenshot.

Student	Date Solved	Status	Attempts	Time Taken	Approach / Notes	Problem
Harshini	17-Nov-2026	Solved	4	10	kmk	Best Time to Buy and Sell Stock
Kaviya	14-Oct-2026	Solved	1	22	3	Climbing Stairs
Harshini	01-Oct-2026	Solved	1	15	Used HashMap	Two Sum
Harshini	02-Oct-2026	Solved	1	12	Used Stack	Valid Parentheses

Table 5.5: Solution Log Entries Visible in the Recent Student Activity Table

5.5 Deluge Scripting

Deluge is used to implement the XP calculation, the update of the student's total and the creation of revision records. The project summary describes three workflow behaviours, listed in Table 5.6. The listings in Appendix C are illustrative Deluge-style descriptions of this logic written for the report and should be checked against the actual scripts.

No.	Workflow	Affected form	Logic summary
1	Calculate XP	Solution_Log	Reads the difficulty of the selected problem and sets XP_Earned to 10 (Easy), 20 (Medium) or 30 (Hard); a failed submission earns 0 XP.
2	Update Total_XP	USER_PROFILE	Adds the calculated XP to the Total_XP of the student.
3	Create revision record	Revision_Tracker	For a successfully solved problem, inserts a record for the same student and problem dated seven days after Date_Solved.

Table 5.6: Deluge Logic and Workflow Summary

5.6 Workflows and Automation

The three workflows act on records of the Solution_Log form and create or update records in USER_PROFILE and Revision_Tracker, as summarised in Table 5.6. The supplied project summary describes what each workflow does but does not include the trigger configuration screens; the trigger events are therefore stated at design level and should be confirmed in the Workflow builder. No Schedule, Blueprint or Approval is part of the completed implementation. Section 8.1 shows how the rules agree with the data captured in the running application.

5.7 Portal and Authentication Pages

A portal named CodeQuest – LeetCode Progress Tracker was created for the application, and four authentication pages were saved in its configuration. Their intended roles are given in Table 5.7. The supplied evidence does not include screenshots of these pages or a recorded sign-up and sign-in test, so the table describes the configuration rather than a tested result. Complete student-specific access and record-level permissions through the portal have not been demonstrated.

Portal page	Purpose
Sign In	Lets a registered portal user enter credentials to open the application.
Sign Up	Lets a new user register for the portal.
Reset Password	Lets a user who has forgotten the password start a password reset.
Create Password	Lets a user set a password for the portal account.

Table 5.7: Portal Authentication Pages

CHAPTER 6

TESTING AND DEBUGGING

6.1 Testing Strategy

Testing should cover individual field rules, lookup selection, the XP calculation, the update of Total_XP, the creation of revision records, the reports and the dashboard. Each test should be performed with valid and invalid data and the result recorded. The cases in Table 6.1 are a practical test plan based on the implemented workflows and requirements. Where the supplied screenshots show the expected behaviour, the status column points to the figure that shows it; this is observation of captured data and not a formal pass. A case is marked as executed only after it has been run in the live application with the result recorded.

6.2 Test Cases

Test ID	Area	Test input / action	Expected result	Status
TC-01	Problem Bank entry	Add a problem with number, name, difficulty, topic and link.	The problem appears in the Problem Bank listing and in lookups.	Listing observed (Fig. 5.1)
TC-02	Lookup selection	Open Solution_Log and choose a student and a problem.	Student name and problem name are displayed in the record and report.	Observed (Fig. 5.5)
TC-03	XP – Easy	Submit a Solved entry for an Easy problem.	XP_Earned is 10.	To be executed
TC-04	XP – Medium	Submit a Solved entry for a Medium problem.	XP_Earned is 20.	To be executed
TC-05	XP – Hard	Submit a Solved entry for a Hard problem.	XP_Earned is 30.	To be executed
TC-06	XP – Failed	Submit a Failed entry for any problem.	XP_Earned is 0.	To be executed
TC-07	Total_XP update	Submit a Solved entry and open the student's profile.	Total_XP increases by the XP earned.	To be executed
TC-08	Revision record	Submit a Solved entry dated 01-Oct-2026.	A revision record dated 08-Oct-2026 is created.	Consistent entries observed (Table 8.1)
TC-09	No revision for failure	Submit a Failed entry.	No seven-day revision record is created.	To be executed
TC-10	Status board	Open the Solution Status Board.	Entries are grouped into Solved and Failed columns.	Observed (Fig. 5.2)

Test ID	Area	Test input / action	Expected result	Status
TC-11	Dashboard KPIs	Compare KPI cards and chart totals with the stored records.	KPI counts and charts match the records.	Partly observed (Table 8.1)
TC-12	Edit a solved entry	Change the status or attempts of an existing entry.	XP and revision records stay correct without duplication.	To be executed
TC-13	Portal pages	Use Sign Up, Sign In, Reset Password and Create Password.	Each page completes its function for a test user.	To be executed

Table 6.1: Test Cases

6.3 Debugging Deluge Scripts

When a workflow does not behave as expected, the developer should first confirm the form event and the order in which the workflows run. The field link names, the lookup values and the handling of empty values should then be verified. The info statement and the workflow execution history help to see what each run did, and unnecessary diagnostic output should be removed before deployment. It should also be checked whether two workflows update the same field and whether the same event can run more than once: a script that adds XP to Total_XP or inserts a revision record must be tested for repeated execution, because an edit of an existing record could otherwise add the XP or create the revision record a second time.

6.4 Defect Log

Defect ID	Potential defect to monitor	Resolution / control	Status
D-01	Editing an existing Solution_Log entry could add XP to Total_XP again.	Recalculate the total from the log, or apply the workflow only when a record is first added.	Verify in application
D-02	Editing a solved entry could create a second revision record.	Check for an existing revision record for the same student, problem and date before inserting.	Verify in application
D-03	A failed entry later corrected to Solved may not update XP or revision.	Run the calculation on edit when the status changes.	Verify in application
D-04	A solution date later than the date of use was found in a sample record (17-Nov-2026 in Fig. 5.5), so no date-range validation is evident.	Add validation that Date_Solved is not in the future.	Verify in application
D-05	The horizontal axis of the topic-wise chart is titled “Problem Name” although it shows counts.	Rename the axis title to indicate the number of problems.	Verify in application

Defect ID	Potential defect to monitor	Resolution / control	Status
D-06	The unit of Time_Taken is not displayed in the report.	State the unit in the field label (for example minutes).	Verify in application
D-07	The same student could log the same problem repeatedly and earn XP each time.	Decide whether repeated solves should earn XP and enforce the rule.	Verify in application

Table 6.2: Defect and Validation Log

6.5 User Acceptance Testing

Acceptance testing should involve the application owner and at least one student following a complete sample session: add problems to the Problem Bank, create student profiles, log a solved problem for each difficulty and one failed attempt, confirm the XP and Total_XP, check that a revision record appears seven days after the solved date, open the reports, the Solution Status Board and the Revision Calendar, and compare the dashboard with the stored records. Acceptance should be based on traceable records, correct totals and clear status groupings. No signed acceptance sheet was supplied with the project files.

CHAPTER 7

DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application should be developed and tested in the development environment of Zoho Creator before it is published to the stage and production environments. A release checklist, given in Table 7.1, should confirm that the forms, lookups, workflows, reports, the dashboard, the portal pages and permissions are configured in the target environment. The screenshots in this report were captured in the running application, whose header shows a trial plan with a limited number of remaining days. A published URL and a deployment record were not supplied with the project files, so the final production deployment remains to be verified.

Deployment check	Verification
Data model	Confirm all four forms, field link names, lookups and difficulty and status values.
Automation	Test the XP, Total_XP and revision workflows with sample records of each difficulty.
Reports and views	Confirm the four reports, the Solution Status Board and the Revision Calendar.
Dashboard	Compare KPI cards and charts with the stored records.
Portal	Test Sign In, Sign Up, Reset Password and Create Password with a test user.
Access	Test each profile with least-privilege permissions.
Recovery	Define export, backup, recovery and change-control procedures.
Release	Record date, version, approver and known limitations.

Table 7.1: Deployment Checklist

7.2 Performance Optimisation

- Use report criteria and the embedded Recent Student Activity table to show only the latest relevant records.
- Avoid repeated queries inside loops; fetch the problem difficulty once for each submitted record.
- Prevent duplicate workflows and repeated updates of the same record, especially for Total_XP.
- Keep dashboard charts focused on decision-relevant indicators such as difficulty and topic.
- Review the number of records displayed in reports as the Solution_Log grows.

7.3 Security Features

Control	Application of the control
Authentication	Access requires sign-in; the portal provides Sign In, Sign Up, Reset Password and Create Password pages.
Authorisation	Role and profile permissions can restrict who adds, edits, views or configures components; custom restrictions are not claimed.
Data minimisation	Forms collect only the fields needed for tracking; the profile stores name, e-mail, goals, user id and total XP.
Validation	Lookups to the profile and problem keep entries linked to existing records; difficulty and status values restrict free text.
No external credentials	The application does not store LeetCode credentials and does not connect to the LeetCode platform.
Privacy	Student details should be shared only with trusted users, and e-mail addresses should not be displayed more widely than necessary.

Table 7.2: Security Controls Implemented

7.4 Governance and Administration

Governance for a student progress tracker is lightweight but still important. Changes should be made in the development environment and promoted through stage to production. Workflow names should describe their purpose, as in Calculate XP and Create revision record. The exported application definition should be stored as a backup before major changes. Permissions and the portal users should be reviewed periodically, and the problem bank should be maintained by the administrator so that difficulty and topic values stay consistent.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

The application covers the practice cycle from problem and student set-up through solution logging, XP calculation, revision scheduling and review through reports, the Solution Status Board, the Revision Calendar and the dashboard. Table 8.1 compares values read from the figures of Chapter 5 with the values expected from the XP and revision rules. It shows consistency between displayed values and is not a substitute for executing the test cases of Chapter 6.

Check	Observed in the screenshots	Expected from the rules
Problems solved	Donut chart segments 16 and 13 (Fig. 5.4); KPI shows 29.	16 + 13 = 29 solved problems.
Total XP	KPI shows 420 (Fig. 5.4).	16 Easy × 10 + 13 Medium × 20 = 420 XP.
Total students	KPI shows 3; Harshini, Kaviya and Arjun appear on the board (Fig. 5.2).	Three students.
Revision – Two Sum	Solved 01-Oct-2026 (Fig. 5.5); calendar shows Two Sum on 8 October (Fig. 5.3).	01-Oct + 7 days = 08-Oct.
Revision – Valid Parentheses	Solved 02-Oct-2026 (Fig. 5.5); calendar shows it on 9 October.	02-Oct + 7 days = 09-Oct.
Revision – Climbing Stairs	Solved 14-Oct-2026 by Kaviya (Fig. 5.5); calendar shows it on 21 October.	14-Oct + 7 days = 21-Oct.

Table 8.1: Cross-Verification of Displayed Values with the XP and Revision Rules

The XP agreement holds when the segment of 16 is Easy and that of 13 is Medium, as the legend suggests. The calendar shows only problem names (Climbing Stairs also appears on 8 and 15 October), so the matches are consistent with the seven-day rule rather than conclusive proof.

8.2 Achievement of Objectives

Objective	Design / deliverable	Assessment
Linked data model	Four forms with lookups from Solution_Log and Revision_Tracker to USER_PROFILE and Problem_Bank.	Covered in the implemented design; Problem Bank and log views captured.
XP and Total_XP	XP rules by difficulty and total update.	Displayed totals consistent with the rule (Table 8.1); individual tests to be executed.

Objective	Design / deliverable	Assessment
Reports and views	Four reports, Solution Status Board and Revision Calendar.	Status Board and Calendar captured (Figs. 5.2 and 5.3).
Dashboard	Three KPI cards, two charts and recent activity.	Captured (Figs. 5.4 and 5.5); the button to All User Profiles was not captured.
Portal pages	Four authentication pages.	Saved; no sign-in test evidence.

Table 8.2: Objective Achievement Mapping

8.3 Comparison with the Existing System

Area	Manual / scattered process	Proposed CodeQuest process
Records	Separate notes, spreadsheets and platform profiles.	Linked records in one centralised application.
Effort measure	No common measure.	XP by difficulty: 10, 20, 30 and 0 for a failed attempt.
Revision	Depends on memory.	A revision record is created seven days after a solved entry.

Table 8.3: Comparison with the Existing System

8.4 Limitations of the Low-Code Approach

The solution depends on platform availability, plan capabilities and the correct configuration of permissions, workflows and the portal. The captured application runs on a trial plan, so plan limits apply. Students enter their activity manually; there is no integration with LeetCode, so the application cannot verify that a problem was solved, and entries depend on honest recording. The revision interval is fixed at seven days, and no Schedule, Blueprint or Approval has been completed. Complete student-specific portal access and record-level permissions were not demonstrated.

The captured screens also show that a sample Solution_Log record carries a date later than the screenshots, suggesting that date validation is not enforced, and that the topic-wise chart titles its horizontal axis “Problem Name” although it shows counts.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Visual builders reduce repetitive coding, while Deluge controls the XP and revision rules, so the project demonstrates a low-code rather than a purely no-code approach. A pro-code service would be justified only if platform-native capabilities become insufficient, for example to import verified activity from a permitted external source.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

CodeQuest – LeetCode Progress Tracker presents a structured low-code solution for recording coding practice, rewarding effort and planning revision. Its four forms connect students, problems, solution attempts and revisions through lookups. The workflows reduce manual effort by calculating XP from problem difficulty, updating each student's total and creating a revision record seven days after a successful solution, while the reports, the Kanban board, the revision calendar and the dashboard give a consolidated view of the progress of the students. Cross-checking the captured screens shows that the displayed totals agree with the XP rule and that revision entries fall seven days after the logged dates.

Schedule, Blueprint, Approval, LeetCode integration and complete student-specific portal access are not implemented and are stated as limitations; the test cases should be executed before the report is used as evidence of a live deployment.

9.2 Future Enhancements

- Make the workflows safe for edits so that XP is never added twice and revision records are never duplicated.
- Add date validation, show the unit of Time_Taken and correct the axis title of the topic-wise chart.
- Use Daily_Goal and Target_Problems in goal indicators and add a scheduled reminder for revisions that fall due.
- Support several revision intervals, student-specific portal access with record-level permissions and a leaderboard.

9.3 Learning Outcomes and Career Relevance

The project gave practical experience in data modelling, lookups, Deluge automation, Kanban and calendar reports, dashboards, portal configuration and test planning, relevant to low-code engineering roles.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help Documentation,” Zoho. Available: https://www.zoho.com/creator/help/ (accessed: [date]).

[2] Zoho Corporation, “Deluge Help,” Zoho. Available: https://www.zoho.com/deluge/help/ (accessed: [date]).

[3] Zoho Corporation, “Workflow Rules in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/ (accessed: [date]).

[4] Zoho Corporation, “Reports in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/ (accessed: [date]).

[5] Zoho Corporation, “Pages in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/ (accessed: [date]).

[6] Zoho Corporation, “Portals in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/ (accessed: [date]).

[7] S. Deterding, D. Dixon, R. Khaled and L. Nacke, “From game design elements to gamefulness: defining ‘gamification’,” in Proc. 15th Int. Academic MindTrek Conf., Tampere, Finland, 2011, pp. 9–15.

[8] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted and D. Rohrer, “Distributed practice in verbal recall tasks: A review and quantitative synthesis,” Psychological Bulletin, vol. 132, no. 3, pp. 354–380, 2006.

[9] OWASP Foundation, “Application Security Verification Standard (ASVS),” OWASP. Available: https://owasp.org/www-project-application-security-verification-standard/ (accessed: [date]).

APPENDIX A – FEATURE COVERAGE CHECKLIST

Feature area	Specified / implemented in project	Evidence to attach / verify
4 forms	Yes	Form builder screenshots and sample records for each form.
Lookup relationships	Yes (Solution_Log and Revision_Tracker)	Form builder screenshots and sample linked records.
Workflows	3 behaviours: XP, Total_XP, revision record	Workflow configuration screenshots and execution logs.
Reports	4 reports	Report screenshots with representative data (Figs. 5.1 and 5.5 partly).
Kanban and calendar views	Solution Status Board and Revision Calendar	Figures 5.2 and 5.3.
Dashboard	3 KPI cards, 2 charts, recent activity, navigation button	Figures 5.4 and 5.5; button not captured.
Portal pages	Sign In, Sign Up, Reset Password, Create Password saved	Page screenshots and a sign-in test.
Schedule	Not implemented	Not applicable.
Blueprint / Approval	Not implemented	Status fields and workflows control the lifecycle.
LeetCode integration	Not implemented (manual entry)	Not applicable.
Deployment	Not evidenced in supplied files	Published URL, release date and deployment checklist.

Table A.1: Feature Coverage Checklist

APPENDIX B – REPORTS AND DASHBOARD CATALOGUE

B.1 Reports

- All_User_Profiles
- All_Problem_Banks
- Solution_Log_Report
- All_Revision_Trackers

B.2 Additional Views

- Solution Status Board (Kanban view of Solution_Log, grouped by status)
- Revision Calendar (calendar view of Revision_Tracker with Month, Week and Day options)

B.3 KPI Cards

- Problems Solved
- Total XP Earned
- Total Students

B.4 Charts and Panels

- Problem Difficulty Distribution (donut chart)
- Topic-wise Solved Problems (horizontal bar chart)
- Recent Student Activity (embedded Solution Log Report)
- All User Profiles navigation button

Frequency	**Monitoring activities (manual; no scheduled workflow is implemented)**
Daily (recommended)	Open the Revision Calendar for revisions that fall due and review the Recent Student Activity table.
Weekly (recommended)	Review the Solution Status Board for Failed entries and compare problems solved and XP on the dashboard.
Monthly (recommended)	Review the difficulty and topic charts, check the Problem Bank for gaps and review the profile list.

Table B.1: Scheduled Monitoring

APPENDIX C – DELUGE LOGIC SUMMARY

The following summaries and code listings describe the behaviour of the CodeQuest workflows. They are illustrative Deluge-style descriptions written for this report from the project summary; they are not copied from the application. The field link names and the form events should be checked against the actual scripts before the listings are reused.

C.1 XP calculation

The difficulty of the selected problem is read through the Problem_Bank lookup. The XP is 10, 20 or 30 for an Easy, Medium or Hard problem and 0 when the submission has failed.

```
difficulty = input.Problem_Bank.Difficulty;
if(input.Status == "Failed")      { xp = 0;  }
else if(difficulty == "Easy")     { xp = 10; }
else if(difficulty == "Medium")   { xp = 20; }
else if(difficulty == "Hard")     { xp = 30; }
input.XP_Earned = xp;
```

Code C.1: *Illustrative XP calculation logic (Solution_Log)*

C.2 Update of Total_XP

The calculated XP is added to the Total_XP of the student whose profile is selected in the record.

```
student = USER_PROFILE[ID == input.USER_PROFILE];
student.Total_XP = student.Total_XP + input.XP_Earned;
```

Code C.2: *Illustrative Total_XP update logic*

C.3 Revision record creation

For a solved attempt, a revision record is inserted with a revision date seven days after the date solved.

```
if(input.Status == "Solved" && input.Date_Solved != null)
{
   insert into Revision_Tracker
   [
      USER_PROFILE = input.USER_PROFILE
      Problem_Bank = input.Problem_Bank
      Revision_Date = input.Date_Solved.addDay(7)
   ]
}
```

Code C.3: *Illustrative revision record creation logic (Revision_Status receives its initial value as configured)*

APPENDIX D – USER MANUAL

D.1 General Access

1. Open the authorised CodeQuest application or portal address using a supported browser.
2. Sign in using the Sign In page; a new user can use Sign Up, and a user who has forgotten the password can use Reset Password.
3. Use the left navigation menu to open User Profiles, Problem Banks, Solution Log, Revision Trackers or the CodeQuest_Dashboard.

D.2 Setting Up

4. Add each problem in Problem Banks and each student in User Profiles with the fields described in Chapter 5.

D.3 Daily Use

5. Open the Solution Log form, choose the student and the problem, and enter the date, status, attempts, time taken and approach notes.
6. Save the record; the XP and the student's total are updated by the workflows, and a revision record is created for a solved problem.
7. Open the Solution Status Board to see Solved and Failed entries.
8. Open the Revision Calendar to see the problems due for revision on each date.

D.4 Review and Administration

9. Open the CodeQuest_Dashboard to see problems solved, total XP, students, charts and recent activity; the administrator should test workflow changes safely and review permissions periodically.

Issue	Recommended check
XP is 0 for a solved problem	Check that the status is Solved and that the problem has a difficulty value in the Problem Bank.
No revision on the calendar	Check that the entry is Solved, that Date_Solved is filled, and that the calendar month shows the date seven days later.

Table D.1: Troubleshooting

APPENDIX E – EVALUATION RUBRIC

Student Name	NITHYA SHREE S	**Register No.**	240701367
Project Title	CodeQuest – LeetCode Progress Tracker	**Date**	11/11/2026

S.No.	**Criterion**	**Indicators**	**Max. Marks**	**Marks Awarded**
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		100	

Table E.1: Evaluation Rubric for the Individual Project

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Table E.2: Performance Level Descriptors

Signature of Internal Examiner **Name:** _____	**Signature of External Examiner** **Name:** _____

APPENDIX F – VIVA-VOCE RECORD SHEET

Student Name	NITHYA SHREE S	**Register No.**	240701367
Department / Section	Computer Science and Engineering	**Date of Viva**	11/11/2026
Application URL Verified	Yes / No	**Demo Duration**	_____min

Q.No.	**Question / Topic Asked**	**Unit**	**Response (E / G / S / NI)**	**Remarks**
1				
2				
3				
4				
5				
6				
7				
8				

Table F.1: Viva-Voce Questions and Responses

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Component	**Max. Marks**	**Marks Awarded**
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	100	

Table F.2: Marks Summary

Examiners' Remarks:

INTERNAL EXAMINER **Name:** _____	**EXTERNAL EXAMINER** **Name:** _____